

Анализ запусков (графики)

```
In [17]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

Т.к. запуск тестов работает порядка 6 часов, используя информацию из `tests_output.md`

Запуск с checker — EXP

```
In [5]: all_n_df = pd.DataFrame([
    [2, 2, 1.0, 1.0, 3.986e-05, 1.734e-05],
    [3, 2, 1.0, 1.0, 3.129e-05, 1.768e-05],
    [4, 2, 1.0, 1.0, 2.718e-05, 1.922e-05],
    [5, 2, 1.0, 1.0, 8.6742e-04, 3.642e-05],
    [4, 3, 1.0, 1.0, 5.153e-05, 1.0279e-04],
    [4, 4, 1.0, 1.0, 6.638e-05, 1.2504e-03],
    [5, 2, 1.0, 1.0, 1.8774e-04, 2.7030e-04],
    [5, 3, 1.029417647, 1.333333333, 6.3620e-04, 2.57705e-03],
    [5, 4, 1.005889359, 1.287142857, 1.3790e-04, 5.93946e-02],
    [5, 5, 1.011944444, 1.2, 2.2970e-04, 1.353693e-03],
    [5, 5, 1.0, 1.0, 1.1239e-04, 1.61852e-03],
    [6, 2, 1.0, 1.0, 9.506e-05, 2.3000e-04],
    [6, 3, 1.054166667, 1.333333333, 1.1699e-04, 2.0387e-03],
    [6, 4, 1.030357143, 1.333333333, 1.7999e-04, 7.70404e-03],
    [6, 5, 1.013944444, 1.2, 2.2970e-04, 1.353693e-02],
    [6, 6, 1.0, 1.0, 1.0956e-04, 1.41867e-02],
    [7, 2, 1.0, 1.0, 7.344e-05, 3.0673e-03],
    [7, 3, 1.066666667, 1.2, 1.0871e-04, 4.69572e-03],
    [7, 4, 1.036694762, 1.333333333, 1.2129e-04, 2.317264e-02],
    [7, 5, 1.000178981, 1.287142857, 1.3790e-04, 5.93946e-02],
    [7, 6, 1.005555556, 1.166666667, 1.4348e-04, 1.1373763e-01],
    [7, 7, 1.0, 1.0, 1.7071e-04, 2.9253281e-01],
    [8, 2, 1.0, 1.0, 1.3846e-04, 8.32133e-04],
    [8, 3, 1.020649206, 1.166666667, 1.6465e-04, 1.774221e-02],
    [8, 4, 1.000357143, 1.1, 1.7402e-04, 1.3062159e-03],
    [8, 5, 1.0917378917, 1.25, 2.0098e-04, 5.5531618e-01],
    [8, 6, 1.033924054, 1.166666667, 1.9679e-04, 2.11556462],
    [8, 7, 1.0047619048, 1.074285714, 2.2531e-04, 2.39140926],
    [8, 8, 1.0, 1.0, 2.2172e-04, 5.6676967],
    [9, 2, 1.0, 1.0, 1.5541e-04, 1.87737e-03],
    [9, 3, 1.05, 1.25, 1.9858e-04, 6.521212e-02],
    [9, 4, 1.075, 1.25, 2.1392e-04, 7.0658728e-01],
    [9, 5, 1.149494949, 1.25, 2.6485e-04, 3.86383524],
    [9, 6, 1.063419934, 1.1428571429, 2.6026e-04, 10.52491967],
    [9, 7, 1.0153846154, 1.0769230769, 2.7640e-04, 22.72462925],
    [9, 8, 1.0, 1.0, 2.5156e-04, 51.93782433],
    [9, 9, 1.0, 1.0, 3.2715e-04, 137.55043566],
], columns=[
    "n", "k", "avg_ratio", "worst_ratio",
    "efficient_time", "checker_time"
])

In [6]: connected_n_df = pd.DataFrame([
    [2, 2, 1.0, 1.0, 2.834e-05, 1.233e-05],
    [3, 2, 1.0, 1.0, 2.771e-05, 1.121e-05],
    [3, 3, 1.0, 1.0, 2.315e-05, 1.329e-05],
    [4, 2, 1.0, 1.0, 3.149e-05, 2.166e-05],
    [4, 3, 1.0, 1.0, 4.328e-05, 6.623e-05],
    [4, 4, 1.0, 1.0, 4.516e-05, 9.402e-05],
    [5, 2, 1.0, 1.0, 5.214e-05, 5.984e-05],
    [5, 3, 1.04719476, 1.333333333, 5.176e-05, 2.725e-04],
    [5, 4, 1.0095238095, 1.2, 5.190e-05, 5.7536e-04],
    [5, 5, 1.0, 1.0, 5.432e-05, 8.9637e-04],
    [6, 2, 1.0, 1.0, 5.917e-05, 1.3028e-04],
    [6, 3, 1.036666667, 1.333333333, 6.634e-05, 1.10963e-03],
    [6, 4, 1.042142857, 1.333333333, 8.5959e-05, 3.96469e-03],
    [6, 5, 1.000333333, 1.166666667, 9.734e-05, 7.23021e-03],
    [6, 6, 1.0, 1.0, 1.166e-04, 1.376305e-02],
], columns=[
    "n", "k", "avg_ratio", "worst_ratio",
    "efficient_time", "checker_time"
])

In [7]: all_n_df = pd.DataFrame([
    [2, 2, 1.0, 1.0, 3.697e-05, 2.765e-05],
    [3, 2, 1.0, 1.0, 3.001e-05, 4.575e-05],
    [3, 3, 1.0, 1.0, 3.228e-05, 3.836e-04],
    [4, 2, 1.0, 1.0, 4.1e-05, 1.5699e-04],
    [4, 3, 1.0, 1.0, 5.341e-05, 2.60632e-03],
    [4, 4, 1.0, 1.0, 6.126e-05, 1.859014e-02],
    [5, 2, 1.0, 1.0, 5.771e-05, 3.0188e-04],
    [5, 3, 1.013884889, 1.333333333, 6.819e-05, 3.92463e-03],
    [5, 4, 1.0, 1.0, 7.54e-05, 2.59594e-02],
    [5, 5, 1.0, 1.0, 1.3219e-04, 1.5663263e-01],
    [6, 2, 1.0, 1.0, 9.74e-05, 5.6314e-04],
    [6, 3, 1.033333333, 1.333333333, 1.3729e-04, 1.066386e-02],
    [6, 4, 1.0, 1.0, 1.1485e-04, 5.90037e-02],
    [6, 5, 1.0, 1.0, 1.1375e-04, 1.3087975e-01],
    [6, 6, 1.0, 1.0, 1.2361e-04, 4.0590394e-01],
], columns=[
    "n", "k", "avg_ratio", "worst_ratio",
    "efficient_time", "checker_time"
])

In [8]: connected_n_df = pd.DataFrame([
    [2, 2, 1.0, 1.0, 4.043e-05, 1.944e-05],
    [3, 2, 1.0, 1.0, 2.834e-05, 1.826e-05],
    [3, 3, 1.0, 1.0, 2.706e-05, 5.734e-05],
    [4, 2, 1.0, 1.0, 3.275e-05, 4.128e-05],
    [4, 3, 1.0, 1.0, 3.384e-05, 1.9159e-04],
    [4, 4, 1.0, 1.0, 4.305e-05, 6.4923e-04],
    [5, 2, 1.0, 1.0, 4.261e-05, 8.686e-05],
    [5, 3, 1.027777778, 1.333333333, 4.522e-05, 6.8516e-04],
    [5, 4, 1.0, 1.0, 4.945e-05, 2.35746e-03],
    [5, 5, 1.0, 1.0, 5.649e-05, 7.19549e-03],
    [6, 2, 1.0, 1.0, 4.824e-05, 1.8263e-04],
    [6, 3, 1.05, 1.333333333, 5.128e-05, 2.10205e-03],
    [6, 4, 1.031578947, 1.25, 8.332e-05, 1.108433e-02],
    [6, 5, 1.0, 1.0, 1.0114e-04, 3.160607e-02],
    [6, 6, 1.0, 1.0, 1.2149e-04, 1.4462731e-01],
], columns=[
    "n", "k", "avg_ratio", "worst_ratio",
    "efficient_time", "checker_time"
])

In [9]: trees_n_df = pd.DataFrame([
    [2, 2, 1.0, 1.0, 6.95e-06, 4.28e-06],
    [3, 2, 1.0, 1.0, 1.409e-05, 7.66e-06],
    [3, 3, 1.0, 1.0, 1.637e-05, 1.11e-05],
    [4, 2, 1.0, 1.0, 2.068e-05, 1.93e-05],
    [4, 3, 1.0, 1.0, 2.330e-05, 5.363e-05],
    [4, 4, 1.0, 1.0, 2.648e-05, 8.116e-05],
    [5, 2, 1.0, 1.0, 2.841e-05, 5.029e-05],
    [5, 3, 1.0, 1.0, 3.222e-05, 2.6301e-04],
    [5, 4, 1.0, 1.0, 3.922e-05, 5.843e-04],
    [5, 5, 1.0, 1.0, 4.031e-05, 9.1937e-04],
    [6, 2, 1.0, 1.0, 3.744e-05, 1.2504e-04],
    [6, 3, 1.0, 1.0, 4.436e-05, 1.12733e-03],
    [6, 4, 1.0, 1.0, 6.893e-05, 4.30549e-03],
    [6, 5, 1.0, 1.0, 7.198e-05, 7.57781e-03],
    [6, 6, 1.0, 1.0, 8.947e-05, 1.39335e-02],
], columns=[
    "n", "k", "avg_ratio", "worst_ratio",
    "efficient_time", "checker_time"
])
```

Запуск только EFFICIENT (замер времени):

```
In [15]: all_time_data = [
    # n, k, time
    (2,2,4.875e-05),
    (3,2,2.893e-05), (3,3,2.5e-05),
    (4,2,3.491e-05), (4,3,3.115e-05), (4,4,3.3e-05),
    (5,2,3.578e-05), (5,3,3.634e-05), (5,4,3.943e-05), (5,5,4.208e-05),
    (6,2,4.457e-05), (6,3,4.575e-05), (6,4,4.944e-05),
    (6,5,9.505e-05), (6,6,7.316e-05),
    (7,2,5.084e-05), (7,3,5.44e-05), (7,4,5.83e-05),
    (7,5,7.07e-05), (7,6,9.761e-05), (7,7,0.00010072),
    (8,2,6.225e-05), (8,3,6.147e-05), (8,4,6.519e-05),
    (8,5,7.096e-05), (8,6,8.081e-05), (8,7,8.851e-05), (8,8,0.142e-03),
    (9,2,7.052e-05), (9,3,7.245e-05), (9,4,7.575e-05),
    (9,5,8.021e-05), (9,6,8.456e-05), (9,7,8.936e-05),
    (9,8,9.332e-05), (9,9,9.756e-05),
    (10,2,7.829e-05), (10,3,7.971e-05), (10,4,0.000143e-03),
    (10,5,6.982e-05), (10,6,9.261e-05), (10,7,9.682e-05),
    (10,8,0.00010213), (10,9,0.00010625), (10,10,0.00010827),
    (11,2,9.987e-05), (11,3,9.36e-05), (11,4,9.702e-05),
    (11,5,0.00010375), (11,6,0.00010655), (11,7,0.00011099),
    (11,8,0.00011804), (11,9,0.00012274), (11,10,0.00012799),
    (11,11,0.00013154),
    (12,2,9.406e-05), (12,3,9.514e-05), (12,4,0.000192e-03),
    (12,5,0.00010452), (12,6,0.00011956), (12,7,0.00013366),
    (12,8,0.00012109), (12,9,0.00012565), (12,10,0.00013052),
    (12,11,0.00013432), (12,12,0.00013939),
    (13,2,0.00011466), (13,3,0.00010872), (13,4,0.0001367),
    (13,5,0.00011792), (13,6,0.00013423), (13,7,0.00013595),
    (13,8,0.00013634), (13,9,0.00014599), (13,10,0.00015699),
    (13,11,0.00015221), (13,12,0.00015459), (13,13,0.00015674),
    (14,2,0.00012165), (14,3,0.00014251), (14,4,0.00012989),
    (14,5,0.00014353), (14,6,0.00013963), (14,7,0.00014581),
    (14,8,0.00015054), (14,9,0.00013758), (14,10,0.00016859),
    (14,11,0.00016904), (14,12,0.00017356), (14,13,0.00017986),
    (14,14,0.00018456),
    (15,2,0.00013598), (15,3,0.00012826), (15,4,0.00013187),
    (15,5,0.00013747), (15,6,0.00014342), (15,7,0.00014973),
    (15,8,0.00015611), (15,9,0.00015358), (15,10,0.00016589),
    (15,11,0.00017571), (15,12,0.00018164), (15,13,0.00018387),
    (15,14,0.00018057), (15,15,0.00018323),
    (16,2,0.00014041), (16,3,0.00013568), (16,4,0.00013683),
    (16,5,0.0001423), (16,6,0.00014748), (16,7,0.00015228),
    (16,8,0.00015976), (16,9,0.00016716), (16,10,0.00017235),
    (16,11,0.00017829), (16,12,0.00018057), (16,13,0.00018191),
    (16,14,0.00019155), (16,15,0.00018951), (16,16,0.00020066),
    ]

connected_time_data = [
    (2,2,4.924e-05),
    (3,2,3.374e-05), (3,3,2.658e-05),
    (4,2,3.611e-05), (4,3,3.121e-05), (4,4,3.333e-05),
    (5,2,3.855e-05), (5,3,3.987e-05),
    (5,4,4.075e-05), (5,5,4.302e-05),
    (6,2,4.01e-05), (6,3,4.038e-05), (6,4,4.263e-05),
    (6,5,4.57e-05), (6,6,4.867e-05),
    (7,2,5.577e-05), (7,3,6.014e-05), (7,4,7.114e-05),
    (7,5,8.741e-05), (7,6,0.00010685), (7,7,0.00011399),
    (8,2,0.00010309), (8,3,0.00010015), (8,4,9.139e-05),
    (8,5,8.791e-05), (8,6,9.016e-05), (8,7,9.488e-05), (8,8,9.588e-05),
    (9,2,6.684e-05), (9,3,6.915e-05), (9,4,7.152e-05),
    (9,5,7.715e-05), (9,6,8.047e-05), (9,7,8.425e-05),
    (9,8,8.877e-05), (9,9,9.053e-05),
    (10,2,7.094e-05), (10,3,7.088e-05), (10,4,7.682e-05),
    (10,5,8.43e-05), (10,6,8.817e-05), (10,7,9.224e-05),
    (10,8,9.725e-05), (10,9,0.00010155), (10,10,0.0001035),
    (11,2,9.08e-05), (11,3,9.041e-05), (11,4,9.071e-05),
    (11,5,9.548e-05), (11,6,0.00010222), (11,7,0.00010909),
    (11,8,0.00011242), (11,9,0.00011569), (11,10,0.00011846),
    (11,11,0.0001212),
    (12,2,9.327e-05), (12,3,9.186e-05), (12,4,9.705e-05),
    (12,5,0.00010428), (12,6,0.00010933), (12,7,0.00011429),
    (12,8,0.00011895), (12,9,0.00012454), (12,10,0.00012873),
    (12,11,0.00013155), (12,12,0.00013276),
    (13,2,0.00010756), (13,3,0.0001153), (13,4,0.00011735),
    (13,5,0.00012111), (13,6,0.00012827), (13,7,0.0001355),
    (13,8,0.00013948), (13,9,0.00014178), (13,10,0.00015059),
    (13,11,0.00015764), (13,12,0.0001638), (13,13,0.0001654),
    (14,2,0.00010688), (14,3,0.00011428), (14,4,0.0001163),
    (14,5,0.00012189), (14,6,0.00012784), (14,7,0.00013407),
    (14,8,0.00013981), (14,9,0.00014652), (14,10,0.00014816),
    (14,11,0.00014764), (14,12,0.00014935), (14,13,0.00014972),
    (14,14,0.00014878),
    (15,2,0.00011309), (15,3,0.00011779), (15,4,0.00015002),
    (15,5,0.00012684), (15,6,0.00013107), (15,7,0.00013709),
    (15,8,0.00014327), (15,9,0.00014933), (15,10,0.00015276),
    (15,11,0.00015335), (15,12,0.00015384), (15,13,0.00015283),
    (15,14,0.00015305), (15,15,0.00015315),
    (16,2,0.00012133), (16,3,0.00012779), (16,4,0.00012993),
    (16,5,0.00013821), (16,6,0.0001493), (16,7,0.00014664),
    (16,8,0.00015239), (16,9,0.00015844), (16,10,0.00016386),
    (16,11,0.00016441), (16,12,0.00016595), (16,13,0.00016517),
    (16,14,0.00016499), (16,15,0.00016447), (16,16,0.00016498),
    ]
```

Графики

```
In [24]: ''' Среднее отношение EFFICIENT '''
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

k_vals = np.array(sorted(all_n_df["k"].unique()))
bound = 2 - 2/k_vals

# ALL (по вершинам)
subset = all_n_df[all_n_df["n"] == n_val]
axes[0, 0].plot(
    subset["avg_ratio"],
    marker="o",
    label=f"n={n_val}"
)

axes[0, 0].axhline(y=1.0, color='black', linestyle='--', alpha=0.4)
axes[0, 0].plot(k_vals, bound, 'r--', linewidth=1.5, alpha=0.6, label=f"2-2/k")
axes[0, 0].set_title("ALL (по вершинам): среднее EFF/OPT")
axes[0, 0].set_xlabel("k")
axes[0, 0].set_ylabel("avg_ratio")
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].legend(ncol=2)

# CONNECTED (по вершинам)
subset = connected_n_df[connected_n_df["n"] == n_val]
axes[0, 1].plot(
    subset["avg_ratio"],
    marker="s",
    label=f"n={n_val}"
)

axes[0, 1].axhline(y=1.0, color='black', linestyle='--', alpha=0.4)
axes[0, 1].plot(k_vals, bound, 'r--', linewidth=1.5, alpha=0.6, label=f"2-2/k")
axes[0, 1].set_title("CONNECTED (по вершинам): среднее EFF/OPT")
axes[0, 1].set_xlabel("k")
axes[0, 1].set_ylabel("avg_ratio")
axes[0, 1].grid(True, alpha=0.3)
axes[0, 1].legend(ncol=2)

# ALL (по ребрам)
subset = all_n_df[all_n_df["n"] == n_val]
axes[1, 0].plot(
    subset["avg_ratio"],
    marker="o",
    label=f"n={n_val}"
)

axes[1, 0].axhline(y=1.0, color='black', linestyle='--', alpha=0.4)
axes[1, 0].plot(k_vals, bound, 'r--', linewidth=1.5, alpha=0.6, label=f"2-2/k")
axes[1, 0].set_title("ALL (по ребрам): среднее EFF/OPT")
axes[1, 0].set_xlabel("k")
axes[1, 0].set_ylabel("avg_ratio")
axes[1, 0].grid(True, alpha=0.3)
axes[1, 0].legend(ncol=2)

# TREES (по ребрам)
subset = trees_df[trees_df["n"] == n_val]
axes[1, 1].plot(
    subset["avg_ratio"],
    marker="d",
    label=f"n={n_val}"
)

axes[1, 1].axhline(y=1.0, color='black', linestyle='--', alpha=0.4)
axes[1, 1].plot(k_vals, bound, 'r--', linewidth=1.5, alpha=0.6, label=f"2-2/k")
axes[1, 1].set_title("TREES (по ребрам): среднее EFF/OPT")
axes[1, 1].set_xlabel("k")
axes[1, 1].set_ylabel("avg_ratio")
axes[1, 1].grid(True, alpha=0.3)
axes[1, 1].legend(ncol=2)

fig.suptitle("Точность EFFICIENT: среднее отношение EFF/OPT по типам графов",
             fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

```
''' Худшее отношение '''
fig, axes2 = plt.subplots(2, 2, figsize=(16, 12))

# ALL (n)
for n_val in sorted(all_n_df["n"].unique()):
    subset = all_n_df[all_n_df["n"] == n_val]
    axes2[0, 0].plot(
        subset["worst_ratio"],
        marker="o",
        label=f"n={n_val}"
    )

axes2[0, 0].axhline(y=1.0, color='black', linestyle='--', alpha=0.4)
axes2[0, 0].plot(k_vals, bound, 'r--', linewidth=1.5, alpha=0.6, label=f"2-2/k")
axes2[0, 0].set_title("ALL (по вершинам): худшее EFF/OPT")
axes2[0, 0].set_xlabel("k")
axes2[0, 0].set_ylabel("worst_ratio")
axes2[0, 0].grid(True, alpha=0.3)
axes2[0, 0].legend(ncol=2)

# CONNECTED (n)
subset = connected_n_df[connected_n_df["n"] == n_val]
axes2[0, 1].plot(
    subset["worst_ratio"],
    marker="s",
    label=f"n={n_val}"
)

axes2[0, 1].axhline(y=1.0, color='black', linestyle='--', alpha=0.4)
axes2[0, 1].plot(k_vals, bound, 'r--', linewidth=1.5, alpha=0.6, label=f"2-2/k")
axes2[0, 1].set_title("CONNECTED (по вершинам): худшее EFF/OPT")
axes2[0, 1].set_xlabel("k")
axes2[0, 1].set_ylabel("worst_ratio")
axes2[0, 1].grid(True, alpha=0.3)
axes2[0, 1].legend(ncol=2)

# ALL (n)
for n_val in sorted(all_n_df["n"].unique()):
    subset = all_n_df[all_n_df["n"] == n_val]
    axes2[1, 0].plot(
        subset["worst_ratio"],
        marker="o",
        label=f"n={n_val}"
    )

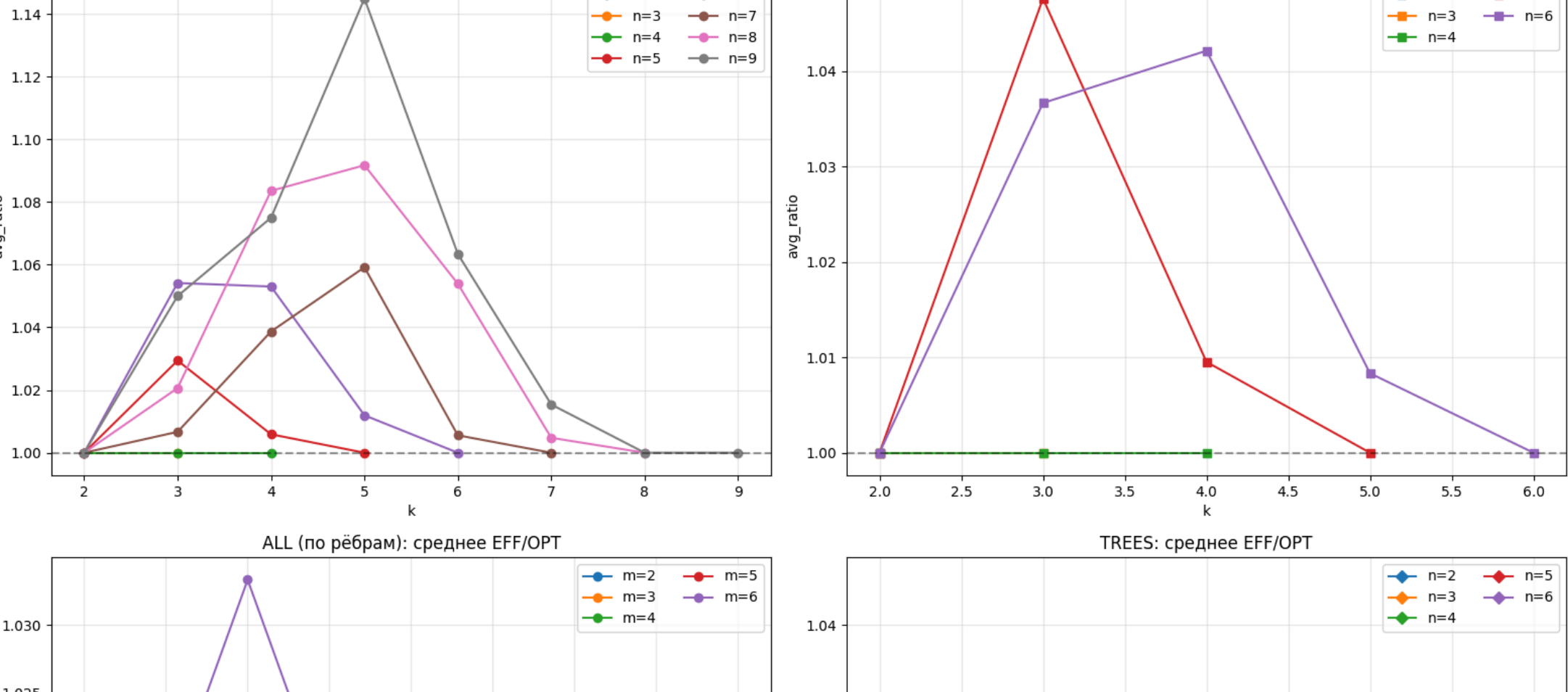
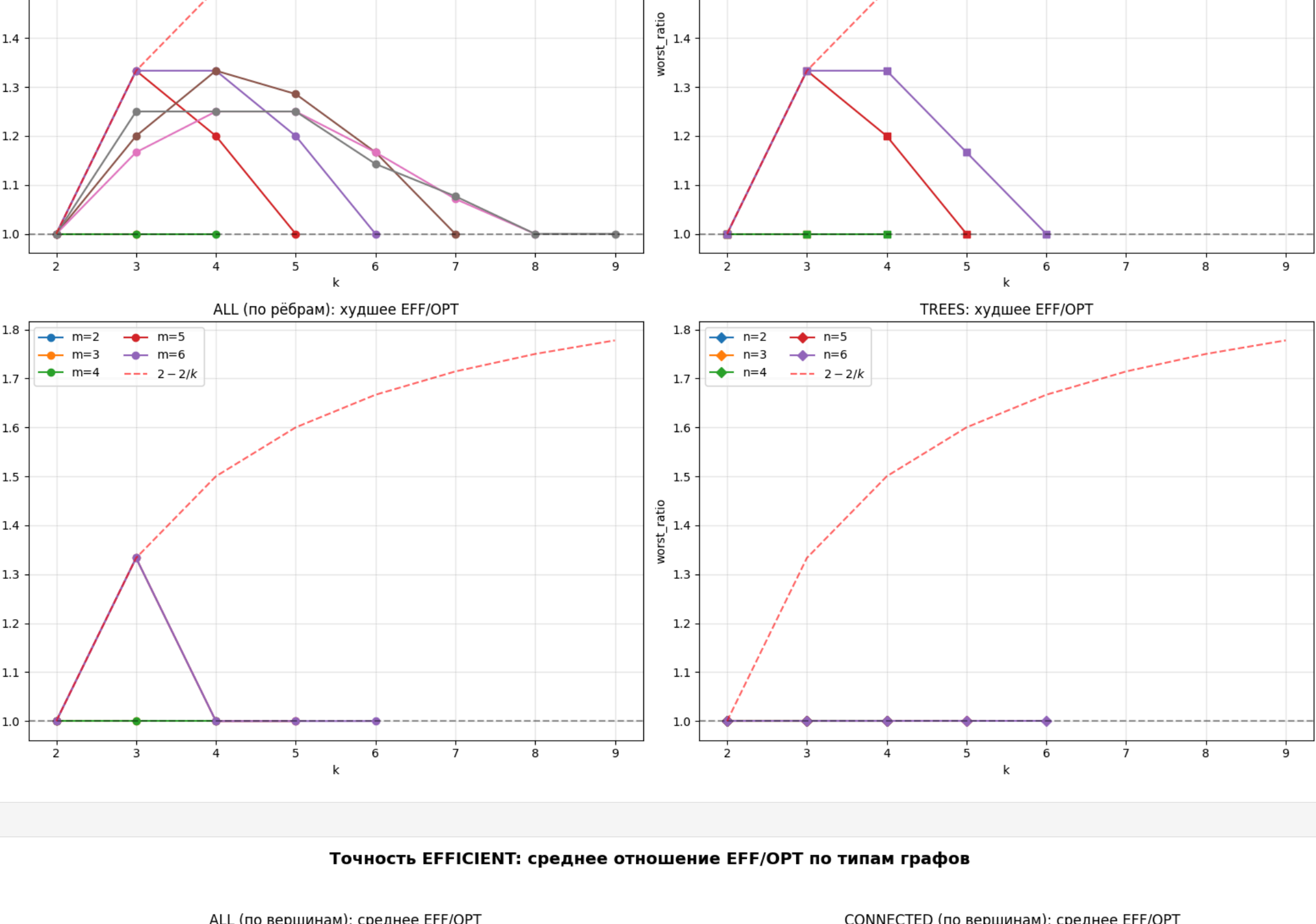
axes2[1, 0].axhline(y=1.0, color='black', linestyle='--', alpha=0.4)
axes2[1, 0].plot(k_vals, bound, 'r--', linewidth=1.5, alpha=0.6, label=f"2-2/k")
axes2[1, 0].set_title("ALL (по ребрам): худшее EFF/OPT")
axes2[1, 0].set_xlabel("k")
axes2[1, 0].set_ylabel("worst_ratio")
axes2[1, 0].grid(True, alpha=0.3)
axes2[1, 0].legend(ncol=2)

# TREES (n)
subset = trees_df[trees_df["n"] == n_val]
axes2[1, 1].plot(
    subset["worst_ratio"],
    marker="d",
    label=f"n={n_val}"
)

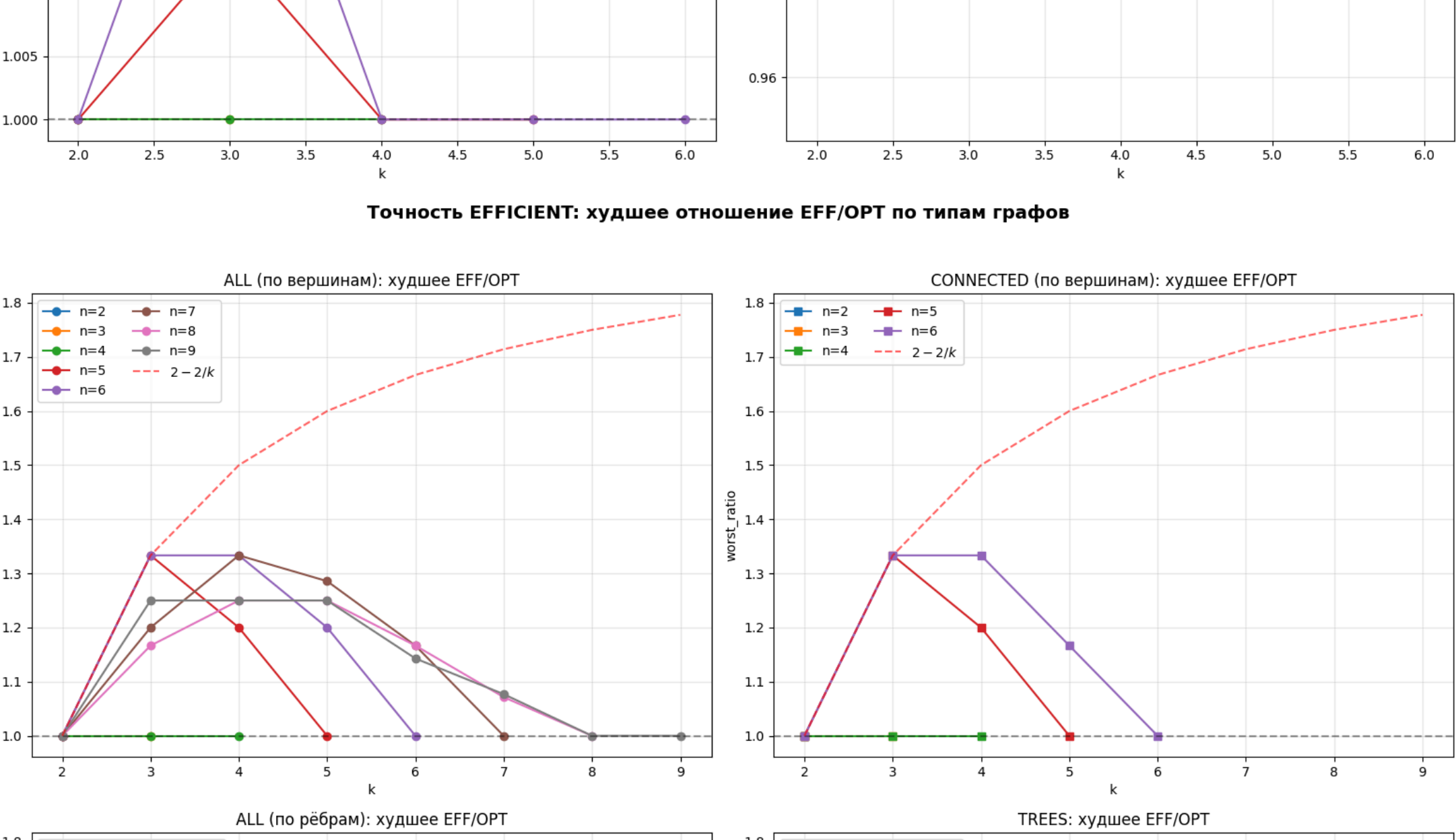
axes2[1, 1].axhline(y=1.0, color='black', linestyle='--', alpha=0.4)
axes2[1, 1].plot(k_vals, bound, 'r--', linewidth=1.5, alpha=0.6, label=f"2-2/k")
axes2[1, 1].set_title("TREES (по ребрам): худшее EFF/OPT")
axes2[1, 1].set_xlabel("k")
axes2[1, 1].set_ylabel("worst_ratio")
axes2[1, 1].grid(True, alpha=0.3)
axes2[1, 1].legend(ncol=2)

fig2.suptitle("Точность EFFICIENT: худшее отношение EFF/OPT по типам графов",
              fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

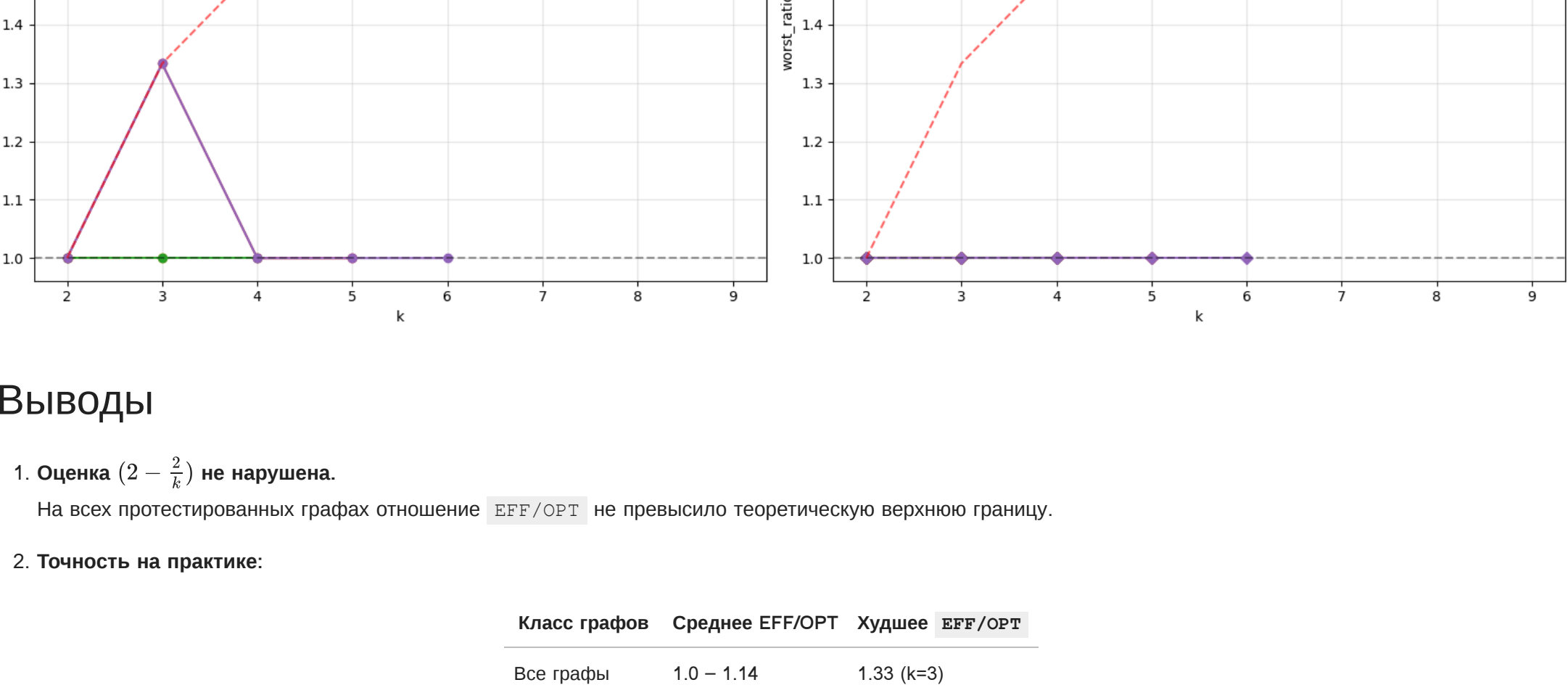
Точность EFFICIENT: среднее отношение EFF/OPT по типам графов



Точность EFFICIENT: худшее отношение EFF/OPT по типам графов



Точность EFFICIENT: худшее отношение EFF/OPT по типам графов



Выводы

1. Оценка $(2 - \frac{2}{k})$ не нарушена.

На всех протестированных графах отношение $\frac{EFF}{OPT}$ не превысило теоретическую верхнюю границу.

2. Точность на практике:

Класс графов	Среднее $\frac{EFF}{OPT}$	Худшее $\frac{EFF}{OPT}$
Все графы	1.0 — 1.14	1.33 (k=3)
Связные графы	1.0 — 1.10	1.33
По числу рёбер	1.0 — 1.11	1.33
Деревья	всегда 1.0	всегда 1.0